

UNIVERSIDAD DEL VALLE DE GUATEMALA
Colegio Universitario



Laboratorio 04

Ernesto Ascencio – 23009
Catedrático: Kimberly Barrera
Sistemas Operativos

15/04/2026
Ciudad de Guatemala, Guatemala

Ejercicio 1

```
root@44b83a88d6a7:/labs# cat /proc/self/maps
aaaad6a90000-aaaad6a96000 r-xp 00000000 00:3e 217728 /usr/bin/cat
aaaad6aa5000-aaaad6aa6000 r--p 00005000 00:3e 217728 /usr/bin/cat
aaaad6aa6000-aaaad6aa7000 rw-p 00006000 00:3e 217728 /usr/bin/cat
aaaada793000-aaaada7b4000 rw-p 00000000 00:00 0 [heap]
ffffa4e0000-ffffa4e30000 rw-p 00000000 00:00 0
ffffa4e30000-ffffa4f0000 r-xp 00000000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffa4f0000-ffffa4fcd000 ---p 0018e000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffa4fcd000-ffffa4fd1000 r--p 0018d000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffa4fd1000-ffffa4fd3000 rw-p 00191000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffa4fd3000-ffffa4fd4000 rw-p 00000000 00:00 0
ffffa4fe8000-ffffa5013000 r-xp 00000000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffa501d000-ffffa501f000 rw-p 00000000 00:00 0
ffffa501f000-ffffa5021000 r--p 00000000 00:00 0 [vvar]
ffffa5021000-ffffa5022000 r-xp 00000000 00:00 0 [vdso]
ffffa5022000-ffffa5024000 r--p 0002a000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffa5024000-ffffa5026000 rw-p 0002c000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffcc1c8000-ffffcc1e9000 rw-p 00000000 00:00 0 [stack]
```

- ¿Qué regiones de memoria identifica (heap, stack, text, shared libraries)?

text	El ejecutable del proceso cargado en RAM	r-xp
data/BSS	Variables globales y estáticas inicializadas/no-inicializadas	r--p / rw-p
heap	Memoria dinámica (malloc/free), crece hacia arriba	rw-p, etiqueta [heap]
stack	Variables locales y marcos de llamada, crece hacia abajo	rw-p, etiqueta [stack]
shared libraries	libc.so, ld-linux.so, etc., mapeadas en el espacio virtual	r-xp / r--p
vvar / vdso	Página del kernel accesible desde usuario para syscalls rápidas	r--p / r-xp

- ¿Qué diferencias observa entre direcciones iniciales y finales?

Se observa que cada fila de `/proc/self/maps` tiene la forma inicio-fin. La diferencia fin - inicio es el tamaño de ese segmento en bytes. Por ejemplo, una entrada como `7f3a00000000-7f3a00021000` ocupa $0x21000 = 132$ KB. El texto del ejecutable suele ocupar pocos KB; libc ocupa varios MB.

- ¿Qué significa que las direcciones no sean contiguas?

El espacio de direcciones virtuales es de 64 bits pero ese proceso solo usa una fracción. El SO gaps intencionales entre regiones por busca aleatorizar las bases de cada región para dificultar los exploits, permitir que el kernel reserve rangos para futuras expansiones y permitir que bibliotecas compartidas mapeen posiciones aleatorias disponibles.

```

root@44b83a88d6a7:/labs# cat /proc/meminfo
MemTotal:      4013480 kB
MemFree:       1763820 kB
MemAvailable:  3285336 kB
Buffers:       98784 kB
Cached:        1443008 kB
SwapCached:    0 kB
Active:        539680 kB
Inactive:      1302312 kB
Active(anon):  300776 kB
Inactive(anon): 0 kB
Active(file):  238904 kB
Inactive(file): 1302312 kB
Unevictable:   0 kB
Mlocked:       0 kB
SwapTotal:     1048572 kB
SwapFree:      1048572 kB
Zswap:         0 kB
Zswapped:      0 kB
Dirty:         0 kB
Writeback:     0 kB
AnonPages:     272812 kB
Mapped:        166220 kB
Shmem:         576 kB
KReclaimable:  167600 kB
Slab:          209728 kB
SReclaimable:  167600 kB
SUnreclaim:    42128 kB
KernelStack:   4592 kB
PageTables:    2420 kB
SecPageTables: 0 kB
NFS_Unstable:  0 kB
Bounce:        0 kB
WritebackTmp:  0 kB
CommitLimit:   3055312 kB
Committed_AS:  1046800 kB
VmallocTotal:  135288315904 kB
VmallocUsed:    5264 kB
VmallocChunk:   0 kB
Percpu:        2144 kB
AnonHugePages: 188416 kB

```

- ¿Qué indicadores permiten conocer el uso real de la memoria?

MemTotal: RAM física.

MemFree: Páginas libres.

MemAvailable: Estimación de RAM disponible.

Cached / Buffers: Páginas ocupadas por la caché del sistema.

SwapTotal / SwapFree: Tamaño y disponibilidad del espacio en disco.

- ¿Qué diferencia hay entre MemFree y MemAvailable?

MemFree muestra el total de páginas sin uso, permite mostrar el caso ya libre y generalmente se activa cuando el kernel tiene mucho cache.

MemAvailable muestra la suma de las páginas libres, cache recuperable y el slab recuperable. Este refleja realmente lo que una app puede usar y siempre lo usa el sistema.

Ejercicio 2

```
→ dockerSistos git:(main) ✕ docker compose exec sistos bash

aaaab4f10000-aaaab4f16000 r-xp 00000000 00:3e 217728 /usr/bin/cat
aaaab4f25000-aaaab4f26000 r--p 00005000 00:3e 217728 /usr/bin/cat
aaaab4f26000-aaaab4f27000 rw-p 00006000 00:3e 217728 /usr/bin/cat
aaaad6f1b000-aaaad6f3c000 rw-p 00000000 00:00 0 [heap]
ffffab43e000-ffffab460000 rw-p 00000000 00:00 0
ffffab460000-ffffab5ee000 r-xp 00000000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffab5ee000-ffffab5fd000 ---p 0018e000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffab5fd000-ffffab601000 r--p 0018d000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffab601000-ffffab603000 rw-p 00191000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffab603000-ffffab60f000 rw-p 00000000 00:00 0
ffffab612000-ffffab63d000 r-xp 00000000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffab647000-ffffab649000 rw-p 00000000 00:00 0
ffffab649000-ffffab64b000 r--p 00000000 00:00 0 [vvar]
ffffab64b000-ffffab64c000 r-xp 00000000 00:00 0 [vdso]
ffffab64c000-ffffab64e000 r--p 0002a000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffab64e000-ffffab650000 rw-p 0002c000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffab650000-ffffab650000 rw-p 00000000 00:00 0 [stack]
root@44b83a88d6a7:/labs/lab4# ./ej2
=== Direcciones de variables ===
globalVar (BSS/data): 0xaaaacf571010 -> valor: 100
localVar (stack): 0xfffff565e18c -> valor: 200
heapPtr (heap): 0xaaaf33f2a0 -> valor: 300

=== /proc/self/maps ===

aaaac8ad0000-aaaac8ad6000 r-xp 00000000 00:3e 217728 /usr/bin/cat
aaaac8ae5000-aaaac8ae6000 r--p 00005000 00:3e 217728 /usr/bin/cat
aaaac8ae6000-aaaac8ae7000 rw-p 00006000 00:3e 217728 /usr/bin/cat
aaab01b21000-aaab01b42000 rw-p 00000000 00:00 0 [heap]
ffffbbace000-ffffbbaf0000 rw-p 00000000 00:00 0
ffffbbaf0000-ffffbbbc7e000 r-xp 00000000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffbbbc7e000-ffffbbbc8d000 ---p 0018e000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffbbbc8d000-ffffbbbc91000 r--p 0018d000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffbbbc91000-ffffbbbc93000 rw-p 00191000 00:3e 218293 /usr/lib/aarch64-linux-gnu/libc.so.6
ffffbbbc93000-ffffbbbc9f000 rw-p 00000000 00:00 0
ffffbbbc9f000-ffffbbbcd000 r-xp 00000000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffbbbcd000-ffffbbbcd9000 rw-p 00000000 00:00 0
ffffbbbcd9000-ffffbbbcd9000 r--p 00000000 00:00 0 [vvar]
ffffbbbcd9000-ffffbbbcd9000 r-xp 00000000 00:00 0 [vdso]
ffffbbbcd9000-ffffbbbcd9000 r--p 0002a000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffbbcd9000-ffffbbcd9000 rw-p 0002c000 00:3e 218275 /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffbbcd9000-ffffbbcd9000 rw-p 00000000 00:00 0 [stack]
root@44b83a88d6a7:/labs/lab4#
```

Parte B

- ¿Las direcciones de memoria se mantienen constantes entre ejecuciones?

No cada vez que se ejecuta las tres direcciones (globalVar, localVar, heapPtr) cambian. El mapa de /proc/self/maps también muestra bases distintas para el ejecutable, heap, stack y bibliotecas.

- ¿Qué fenómeno del sistema operativo explica este comportamiento?

Address Space Layout Randomization (ASLR) porque el kernel realiza de manera aleatoria la dirección base de cada región de memoria en cada nuevo proceso.

- ¿Qué ventaja ofrece este mecanismo?

Entre muchas ventajas las principales son la defensa contra exploits de memoria, ataques como return-to-libc, o Return Oriented Programming que requieren conocer direcciones específicas del sistema .

Parte C

- ¿En qué región se encuentra la variable global?

GlobalVar: Global inicializada en la region del mapa data segment (BSS/data), se identifica en /proc/self/maps con el rango del ejecutable con permisos rw-p, sin etiqueta especial

- ¿En qué región se encuentra la variable local?

localVar : Local (automática) en la region del mapa stack , se identifica en /proc/self/maps con la región etiquetada [stack], dirección alta

- ¿En qué región se encuentra la memoria dinámica (malloc)?

heapPtr : Dinámica (malloc) en la region del mapa heap, se identifica en /proc/self/maps con la región etiquetada [heap], dirección intermedia

- ¿Cómo puede identificar cada región dentro del mapa?

heap: etiqueta explícita, permisos rw-p, dirección por encima del segmento de datos.

stack: etiqueta explícita, dirección más alta del espacio de usuario.

Texto del ejecutable: primera entrada, permisos r-xp, ruta al binario.

Librerías: entradas con ruta /lib/ o /usr/lib/, permisos r-xp.

- ¿Qué patrón observa en los rangos de direcciones?

Se observa que siempre está en orden de menor a mayor dirección:

“texto (código) < data/BSS < [heap] ↑ ...gap... ↓ librerías ...gap... [stack]”

El heap crece hacia direcciones altas (↑) y el stack crece hacia direcciones bajas (↓).

Ejercicio 3

```
→ dockerSistos git:(main) * docker compose exec sistos bash
Reservando 50 MB...
Memoria reservada. Iniciando accesos cada 4096 bytes (1 pagina)
...
Monitorea con 'top' o 'htop' en otra terminal.

Paginas accedidas: 1000 / 12800
Paginas accedidas: 2000 / 12800
Paginas accedidas: 3000 / 12800
Paginas accedidas: 4000 / 12800
Paginas accedidas: 5000 / 12800
Paginas accedidas: 6000 / 12800
Paginas accedidas: 7000 / 12800
Paginas accedidas: 8000 / 12800
Paginas accedidas: 9000 / 12800
Paginas accedidas: 10000 / 12800
Paginas accedidas: 11000 / 12800
Paginas accedidas: 12000 / 12800

Total de paginas accedidas: 12800
Memoria total tocada: 50 MB
root@44b83a88d6a7:/labs/lab4# ./eje3
bash: ./eje3: No such file or directory
root@44b83a88d6a7:/labs/lab4# ./ej3
Reservando 50 MB...
Memoria reservada. Iniciando accesos cada 4096 bytes (1 pagina)
...
Monitorea con 'top' o 'htop' en otra terminal.

Paginas accedidas: 1000 / 12800
Paginas accedidas: 2000 / 12800
Paginas accedidas: 3000 / 12800
Paginas accedidas: 4000 / 12800
Paginas accedidas: 5000 / 12800
Paginas accedidas: 6000 / 12800
Paginas accedidas: 7000 / 12800
Paginas accedidas: 8000 / 12800
Paginas accedidas: 9000 / 12800
Paginas accedidas: 10000 / 12800
Paginas accedidas: 11000 / 12800
Paginas accedidas: 12000 / 12800

Total de paginas accedidas: 12800
Memoria total tocada: 50 MB
root@44b83a88d6a7:/labs/lab4#
```

```
0[|] 0.7% 4[|] 0.0%
1[|] 0.0% 5[|] 0.7%
2[|] 0.0% 6[|] 0.0%
3[|] 0.0% 7[|] 0.0%
Mem[|||||] 422M/3.83G Tasks: 4, 0 thr; 1 running
Swp[|] 0K/1024M Load average: 0.18 0.11 0.
Uptime: 00:22:30
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+
116	root	20	0	4288	2920	2408	R	0.7	0.1	0:00.07
1	root	20	0	4140	2912	2656	S	0.0	0.1	0:00.04
9	root	20	0	4140	3228	2844	S	0.0	0.1	0:00.29
104	root	20	0	4140	3296	2784	S	0.0	0.1	0:00.06

```
F1Help F2Setup F3SearchF4FilterF5Tree F6SortByF7Nice -F8Nice
```

```
Paginas accedidas: 9000 / 12800
Paginas accedidas: 10000 / 12800
Paginas accedidas: 11000 / 12800
Paginas accedidas: 12000 / 12800

Total de paginas accedidas: 12800
Memoria total tocada: 50 MB
root@44b83a88d6a7:/labs/lab4# ./ej3
Reservando 50 MB...
Memoria reservada. Iniciando accesos cada 4096 bytes (1 pagina)
...
Monitorea con 'top' o 'htop' en otra terminal.

Paginas accedidas: 1000 / 12800
Paginas accedidas: 2000 / 12800
Paginas accedidas: 3000 / 12800
Paginas accedidas: 4000 / 12800
Paginas accedidas: 5000 / 12800
Paginas accedidas: 6000 / 12800
Paginas accedidas: 7000 / 12800
Paginas accedidas: 8000 / 12800
Paginas accedidas: 9000 / 12800
Paginas accedidas: 10000 / 12800
Paginas accedidas: 11000 / 12800
Paginas accedidas: 12000 / 12800

Total de paginas accedidas: 12800
Memoria total tocada: 50 MB
root@44b83a88d6a7:/labs/lab4# ./ej3
Reservando 50 MB...
Memoria reservada. Iniciando accesos cada 4096 bytes (1 pagina)
...
Monitorea con 'top' o 'htop' en otra terminal.

Paginas accedidas: 1000 / 12800
Paginas accedidas: 2000 / 12800
Paginas accedidas: 3000 / 12800
Paginas accedidas: 4000 / 12800
Paginas accedidas: 5000 / 12800
Paginas accedidas: 6000 / 12800
Paginas accedidas: 7000 / 12800
Paginas accedidas: 8000 / 12800
Paginas accedidas: 9000 / 12800
Paginas accedidas: 10000 / 12800
Paginas accedidas: 11000 / 12800
Paginas accedidas: 12000 / 12800
```

```
0[|] 4.7% 4[|] 0.7%
1[|] 2.0% 5[|] 0.7%
2[|] 1.3% 6[|] 2.0%
3[|] 0.7% 7[|] 0.7%
Mem[|||||] 445M/3.83G Tasks: 5, 0 thr; 2 running
Swp[|] 0K/1024M Load average: 0.43 0.18 0.
Uptime: 00:23:40
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+
116	root	20	0	4288	2920	2408	R	0.7	0.1	0:00.27
1	root	20	0	4140	2912	2656	S	0.0	0.1	0:00.04
9	root	20	0	4140	3228	2844	S	0.0	0.1	0:00.37
104	root	20	0	4140	3296	2784	S	0.0	0.1	0:00.06
141	root	20	0	53412	5320	1140	R	0.0	0.1	0:00.00

```
F1Help F2Setup F3SearchF4FilterF5Tree F6SortByF7Nice -F8Nice
```

- ¿Observa cambios en el uso de memoria?

Sí, con htop, la columna RES (memoria residente = páginas físicamente en RAM) crece de forma gradual y visible durante la ejecución, aunque VIRT (virtual) ya muestra los 50 MB desde el malloc.

- ¿Qué relación tiene el tamaño de acceso con el tamaño de página?

4096 bytes = 4 KB = exactamente el tamaño de una página en x86-64 (Linux). Al acceder `arr[i * 4096]`, se garantiza que cada acceso cae en una página distinta. Esto significa que:

1. Cada acceso toca exactamente una nueva página generando exactamente un page fault por iteración.
2. Si se accediera cada <4096 bytes, muchos accesos caerían en la misma página ya cargada → menos page faults, menos actividad observable.
3. Si se accediera cada >4096 bytes, se saltaría una página entera y se cargarían menos páginas y quedarían "huecos" sin mapear físicamente.

Ejercicio 4

```
root@44b83a88d6a7:/labs/lab4# ./ej4
=== Fase 1: Reservando 500 MB con malloc ===

--- Estado de memoria [despues de malloc, antes de usar] ---
VmPeak: 2912 kB
VmSize: 2904 kB
VmRSS: 1400 kB

=== Fase 2: Esperando 5s sin tocar la memoria ===
(Observa vmstat: sin page faults aun)

--- Estado de memoria [despues de esperar, antes de acceder] ---
VmPeak: 2912 kB
VmSize: 2904 kB
VmRSS: 1412 kB

=== Fase 3: Accediendo cada 4096 bytes (genera page faults) ===
Progreso: 0% (0 paginas)
Progreso: 7% (10000 paginas)
Progreso: 15% (20000 paginas)
Progreso: 23% (30000 paginas)
Progreso: 31% (40000 paginas)
Progreso: 39% (50000 paginas)
Progreso: 46% (60000 paginas)
Progreso: 54% (70000 paginas)
Progreso: 62% (80000 paginas)
Progreso: 70% (90000 paginas)
Progreso: 78% (100000 paginas)
Progreso: 85% (110000 paginas)
Progreso: 93% (120000 paginas)

--- Estado de memoria [despues de acceder toda la memoria] ---
VmPeak: 2912 kB
VmSize: 2904 kB
VmRSS: 1352 kB

=== Fin: toda la memoria fue usada ===
root@44b83a88d6a7:/labs/lab4#
```

```
dockerSistos git:(main) x docker compose exec sistos bash
0 0 0 1859684 98908 1611508 0 0 36 0 1565 1951 3 1 96 0 0
0 0 0 1856940 98924 1611492 0 0 0 0 1555 1896 4 1 96 0 0
procs memory swap io system cpu
r b swpd free buff cache si so bi bo in cs us sy id wa st
0 0 0 1864356 98924 1611492 0 0 0 0 1426 2035 1 1 98 0 0
0 0 0 1864356 98924 1611492 0 0 0 0 299 383 0 0 100 0 0
0 0 0 1864356 98924 1611492 0 0 0 0 105 111 0 0 100 0 0
1 0 0 1863600 98924 1611492 0 0 0 0 272 350 0 0 100 0 0
1 0 0 1863600 98936 1611492 0 0 0 44 189 225 0 0 99 0 0
0 0 0 1865656 98936 1611492 0 0 0 0 168 136 0 0 100 0 0
0 0 0 1865404 98936 1611492 0 0 0 0 391 233 2 0 98 0 0
0 0 0 1865152 98936 1611492 0 0 0 0 80 103 0 0 100 0 0
0 0 0 1865404 98936 1611492 0 0 0 0 132 179 0 0 100 0 0
0 0 0 1864912 98936 1611492 0 0 0 0 180 231 0 0 100 0 0
0 0 0 1864912 98936 1611492 0 0 0 0 1798 2350 2 1 97 0 0
0 0 0 1864912 98936 1611492 0 0 0 0 309 384 0 0 100 0 0
1 0 0 1864912 98936 1611492 0 0 0 0 175 165 0 0 99 0 0
0 0 0 1864912 98936 1611492 0 0 0 0 292 393 0 0 100 0 0
0 0 0 1864152 98936 1611544 0 0 0 0 467 614 0 0 100 0 0
0 0 0 1864152 98936 1611544 0 0 0 0 62 84 0 0 100 0 0
0 0 0 1864152 98936 1611544 0 0 0 0 94 118 0 0 100 0 0
0 0 0 1864152 98936 1611544 0 0 0 0 132 173 0 0 100 0 0
0 0 0 1865064 98936 1611544 0 0 0 0 131 172 0 0 100 0 0
0 0 0 1864816 98936 1611544 0 0 0 0 1162 1247 0 4 96 0 0
0 0 0 1864816 98936 1611544 0 0 0 0 1636 2374 1 1 98 0 0
0 0 0 1864816 98936 1611544 0 0 0 0 475 658 0 0 100 0 0
0 0 0 1864816 98936 1611544 0 0 0 0 195 292 0 0 100 0 0
0 0 0 1864568 98936 1611544 0 0 0 0 279 409 0 0 100 0 0
0 0 0 1864568 98936 1611544 0 0 0 0 109 135 0 0 100 0 0
0 0 0 1864568 98936 1611544 0 0 0 0 116 163 0 0 100 0 0
0 0 0 1865144 98936 1611544 0 0 0 0 1110 1228 0 4 96 0 0
0 0 0 1865144 98936 1611544 0 0 0 0 159 208 0 0 100 0 0
0 0 0 1865144 98936 1611544 0 0 0 0 183 251 0 0 100 0 0
0 0 0 1865144 98936 1611544 0 0 0 0 317 430 0 0 100 0 0
0 0 0 1864676 98936 1611544 0 0 0 0 1771 2327 3 1 96 0 0
0 0 0 1864676 98936 1611544 0 0 0 0 251 324 0 0 100 0 0
0 0 0 1864676 98936 1611544 0 0 0 0 138 194 0 0 100 0 0
0 0 0 1864676 98936 1611544 0 0 0 0 178 247 0 0 100 0 0
0 0 0 1895000 98936 1611544 0 0 0 0 1222 1319 0 5 95 0 0
0 0 0 1895000 98936 1611544 0 0 0 32 84 70 0 0 100 0 0
0 0 0 1892504 98936 1611544 0 0 0 0 158 139 0 0 100 0 0
1 0 0 1892504 98936 1611544 0 0 0 0 258 325 0 0 100 0 0
0 0 0 1887652 98936 1611544 0 0 0 0 2723 4208 2 1 96 0 0
```

Parte C

- ¿Qué diferencia observa entre reservar memoria con malloc y acceder realmente a esa memoria?

Malloc solo reserva espacio virtual, actualiza la estructura del heap y el mapa de memoria del proceso, pero no asigna nada en RAM. Solo cuando el programa escribe

en una dirección de esa región, el hardware genera un page fault y el kernel asigna un marco físico real.

- ¿Qué ocurre cuando el programa escribe por primera vez en una nueva región de 4096 bytes?
 1. La CPU intenta escribir pero la página no tiene marco físico asignado entonces genera una excepción de hardware (page fault).
 2. El kernel intercepta la excepción, luego busca un marco libre en la RAM física, lo inicializa a ceros (requisito de seguridad POSIX) y actualiza la tabla de páginas del proceso.
 3. La CPU reintenta la instrucción, ahora con la página mapeada y finalmente la escritura tiene éxito y el proceso continúa.
- ¿Qué son los page faults y por qué se producen en este ejercicio?

Un page fault es una excepción del hardware que ocurre cuando una dirección virtual no tiene una traducción válida a dirección física en la tabla de páginas. Se producen en el ejercicio porque malloc crea entradas "reservadas, pero no mapeadas"

El primer acceso a cada una de esas páginas dispara el fault. En vmstat 1 se observa como la columna pgflt o flt se dispara durante la Fase 3 y

permanece en 0 durante la Fase 2.

- ¿Por qué no toda la memoria reservada se utiliza físicamente desde el instante en que malloc retorna?

Linux permite que los programas pidan muchísima memoria virtual aunque no haya tanta RAM física disponible. El sistema acepta estas peticiones porque la mayoría de los procesos reservan espacio "por si acaso" pero casi nunca lo usan todo al mismo tiempo.

- ¿Qué relación tiene el acceso cada 4096 bytes con el concepto de página de memoria?

Tiene relación porque Linux no entrega la memoria real por bloques grandes, sino por **páginas** individuales de 4 KB.

Cuando el programa toca, aunque sea un solo byte de una página (como el primero), el sistema operativo se da cuenta de que esa "promesa" de memoria virtual ahora necesita un lugar físico real. Como no puede entregar menos de una página completa (4096 bytes), el **kernel** se detiene, busca un espacio en la RAM y lo asigna.

